

// Sort theo chuỗi

```
const sortedByNameAZ = useMemo(() => {  
  
    return [...products].sort((a, b) =>  
  
        a.title.localeCompare(b.title, "vi", { sensitivity: "base",  
        numeric: true })  
  
    );  
}, [products]);
```

// Sort theo số

```
const sortedPriceAsc = useMemo(() => {  
  
    return [...products].sort((a, b) => a.price - b.price);  
  
}, [products]);
```

// Sort theo thời gian

```
const sortedByNewest = useMemo(() => {  
  
    return [...products].sort(  
  
        (a, b) => new Date(b.createdAt).getTime() - new  
        Date(a.createdAt).getTime()  
  
    );  
}, [products]);
```

// Sort theo boolean

```
const sortedByDoneFalseFirst = useMemo(() => {  
  
    return [...todos].sort((a, b) => Number(a.isDone) -  
    Number(b.isDone));  
  
}, [todos]);
```

// Sort nhiều tiêu chí: DESC số, ASC chữ

```
const sortedByIdDescThenName = useMemo(() => {  
  
    return [...products].sort((a, b) => {  
  
        const idDiff = Number(b.id) - Number(a.id);  
  
        if (idDiff !== 0) return idDiff;  
  
        return a.title.localeCompare(b.title, "vi", { sensitivity:  
"base", numeric: true });  
  
    });  
}, [products]);
```

// DESC ngày, ASC giá

```
const sortedByNewestThenPriceAsc = useMemo(() => {  
  
    return [...products].sort((a, b) => {  
  
        const timeDiff = new Date(b.createdAt).getTime() -  
        new Date(a.createdAt).getTime();  
  
        if (timeDiff !== 0) return timeDiff;  
  
        return (a.price ?? 0) - (b.price ?? 0);  
  
    });  
}, [products]);
```

// Trạng thái custom → Cập nhật mới trước

```
const statusOrder = { pending: 0, processing: 1, shipped:  
2, done: 3 } as const;  
  
const sortedByStatusThenUpdated = useMemo(() => {  
  
    return [...orders].sort((a, b) => {  
  
        const stDiff = statusOrder[a.status] -  
        statusOrder[b.status];  
  
        if (stDiff !== 0) return stDiff;  
  
        return new Date(b.updatedAt).getTime() - new  
        Date(a.updatedAt).getTime();  
  
    });  
}, [orders]);
```

```
});  
}, [orders]);  
  
// isDone: false trước → Tên A→Z  
  
const sortedUndoneFirstThenName = useMemo(() => {  
  
    return [...todos].sort((a, b) => {  
  
        const doneDiff = Number(a.isDone) -  
        Number(b.isDone);  
  
        if (doneDiff !== 0) return doneDiff;  
  
        return a.title.localeCompare(b.title, "vi", { sensitivity:  
"base", numeric: true });  
  
    });  
}, [todos]);
```

// Độ dài tên tăng dần → ID số DESC

```
const sortedByTitleLenThenId = useMemo(() => {  
  
    return [...products].sort((a, b) => {  
  
        const lenDiff = (a.title?.length ?? 0) - (b.title?.length ??  
0);  
  
        if (lenDiff !== 0) return lenDiff;  
  
        return Number(b.id) - Number(a.id);  
});  
}, [products]);
```

// Native Stack Navigation

```
const Stack =  
createNativeStackNavigator<RootStackParamList>();  
  
<NavigationContainer>  
  
    <Stack.Navigator initialRouteName="Home">  
  
        <Stack.Screen name="Home"  
        component={HomeScreen} />  
  
    </Stack.Navigator>  
  
</NavigationContainer>
```

```
type Props =  
NativeStackScreenProps<RootStackParamList, "Home">;  
  
export const HomeScreen = ({ navigation, route }: Props)  
=> {}
```

// Bottom Tab Navigation

```
const Tab =  
createBottomTabNavigator<ProfileTabParamList>();  
  
<Tab.Navigator  
  
    initialRouteName="UserInfo"  
  
    screenOptions={{  
  
        headerShown: false,  
  
        tabBarActiveTintColor: "green",  
  
        tabBarInactiveTintColor: "gray",  
  
    }}  
  
>  
  
    <Tab.Screen name="UserInfo"  
    component={UserInfoScreen} />  
  
    <Tab.Screen name="ClassInfo"  
    component={ClassInfoScreen} />  
  
</Tab.Navigator>  
  
type Props =  
BottomTabScreenProps<ProfileTabParamList,  
"UserInfo">;  
  
export const UserInfoScreen = ({}: Props) => {}
```

// Drawer Navigation

```
const Drawer =  
createDrawerNavigator<RootDrawerParamList>();  
  
<NavigationContainer>  
  
    <Drawer.Navigator  
  
        initialRouteName="Home"  
  
        screenOptions={{  
  
            headerShown: true,  
  
            drawerActiveTintColor: "green",  
  
            drawerInactiveTintColor: "gray",  
  
        }}  
  
    >  
  
        <Drawer.Screen name="Home"  
        component={HomeScreen} />  
  
        <Drawer.Screen name="Settings"  
        component={SettingsScreen} />  
  
    </Drawer.Navigator>  
  
</NavigationContainer>
```

```
type Props = DrawerScreenProps<DrawerParamList,  
"Home">;  
  
export function HomeScreen = ({ navigation }: Props) =>  
{}
```

Install

```
npx expo install @react-navigation/native react-native-  
screens react-native-safe-area-context @react-  
navigation/native-stack @react-navigation/bottom-tabs  
@react-navigation/drawer react-native-paper react-dom  
react-native-web  
  
npx expo install react-native-gesture-handler react-native-  
reanimated react-native-worklets
```

Refresh Control

```
<RefreshControl  
  
    refreshing={isRefreshing}  
  
    onRefresh={handleRefresh}  
  
></RefreshControl>
```

useFetch

```
export const useFetch = (baseUrl: string) => {  
  
    const [isLoading, setIsLoading] = useState(false);  
  
    const request = async <T>(  
  
        url: string,  
  
        options: RequestInit  
  
    ): Promise<T[]> => {  
  
        setIsLoading(true);  
  
        const res = await fetch(baseUrl + url, {  
  
            headers: { "Content-Type": "application/json" },  
  
            ...options,  
  
        });  
  
        setIsLoading(false);  
  
        if (!res.ok) return [];  
  
        return res.json();  
  
    };  
};
```

```
const get = <T>(url: string) => request<T>(url, {
method: "GET" });

const post = (url: string, data: any) =>

    request(url, { method: "POST", body:
JSON.stringify(data) });

const put = (url: string, data: any) =>

    request(url, { method: "PUT", body:
JSON.stringify(data) });

const del = <T>(url: string) => request<T>(url, {
method: "DELETE" });

return { isLoading, get, post, put, del };

};

// List
<List.Section title="Product Details">

    <List.Item title="ID" description={data.id} />

    <List.Item title="Title" description={data.title} />

    <List.Item title="Price"
description={data.price.toString()} />

    <List.Item title="Description"
description={data.description} />

</List.Section>

// Handle Change Input

const handleChangeInput = (value: string, key: string) => {
{

    setFormData((prev) => {

        return {

            ...prev,

            [key]: key === "price" ? Number(value) : value,

        };

    });

};

};

// UseRef

inputRef.current?.clear();

inputRef.current?.focus();

const X = useRef<ReturnType<typeof setTimeout> |
null>(null);

// Cuộn list đến item mới nhất:

const listRef = useRef<FlatLis>(tnull);

listRef.current?.scrollToIndex({ index: 0, animated: true
});

// Flat List bổ sung

<FlatList

    ref={listRef}

    data={sortedProducts}

    keyExtractor={(it) => it.id}

    renderItem={({ item }) => (

        <ProductCard

            onUpdate={handleUpdate}

            onViewDetailsBtn={handleViewDetails}

            onDeleteBtn={handleDelete}

            data={item}
```

```
/>

    )}

    ListHeaderComponent={() => (

        <Card style={{ margin: 10 }}><Card.Title title="Danh
sách sản phẩm" /></Card>

    )}

    ItemSeparatorComponent={() => <View style={{
height: 8 }} />}

    ListEmptyComponent={() => (

        <Card style={{ margin: 10 }}>

            <Card.Content><Text>Chưa có sản phẩm
nào</Text></Card.Content>

        </Card>

    )}

/>

// Sort nâng cao

const [q, setQ] = useState("");

const [sortBy, setSortBy] = useState<"priceDesc" |
"idDesc" | "titleAZ">("priceDesc");

const visibleProducts = useMemo() => {

    let data = products.filter(p =>

        p.title.toLowerCase().includes(q.toLowerCase().trim())

    );

    if (sortBy === "priceDesc") {

        data.sort((a, b) => b.price - a.price || Number(b.id) -
Number(a.id));

    } else if (sortBy === "idDesc") {

        data.sort((a, b) => Number(b.id) - Number(a.id));

    } else {

        data.sort((a, b) => a.title.localeCompare(b.title, "vi", {
sensitivity: "base", numeric: true }));

    }

    return data;

}, [products, q, sortBy]);

<Card.Content>

    <TextInput

        style={styles.inputContainer}

        placeholder="Tìm theo tên..."

        onChangeText={setQ}

    />

    <Button onPress={() => setSortBy(s => s ===
"priceDesc" ? "titleAZ" : s === "titleAZ" ? "idDesc" :
"priceDesc")}>

        Đổi sắp xếp: {sortBy}

    </Button>

</Card.Content>

Import

import { NavigationContainer } from "@react-
navigation/native";

import { createNativeStackNavigator } from "@react-
navigation/native-stack";

import { StatusBar } from "expo-status-bar";
```

```
import { StyleSheet, Text, View } from "react-native";

import { SafeAreaView } from "react-native-safe-
area-context";

import { NativeStackScreenProps } from "@react-
navigation/native-stack";

import { StatusBar } from "expo-status-bar";
```